```
function [Nref,final_althyp_index,network_flag,removed_points_flag] =
ps_spatial_unwrapping(ens_coh_threshold,varfac_threshold,ref_cn,Nifgs,Npsc,Npsc_
selections,Nref,Narcs,Btemp,Bdop,final_model,std_param,sig2_est,breakpoint,break
point2,network_flag,z)

% Function to spatially unwrap the phase ambiguities.
%
% Input:    - ens_coh_threshold   ensemble coherence threshold
%           - ref_cn              coordinates reference point
%           - Nifgs               number of interferograms
%           - Npsc                number of persistent scatterer candidates
%           - Npsc_selections     number of psc selections
%           - Nref                number of reference points
%           - Narcs               number of arcs
%           - Btemp               temporal baselines [year]
%           - Bdop                Doppler baselines [Hz]
%           - final_model         model used for unwrapped data
%           - std_param           standard deviations for pseudo-
%                                 observations
%           - sig2_est            estimated variance components
%           - breakpoint          breakpoint in case of double
%                                 linear model
%           - breakpoint2         second breakpoint, should be larger
%                                 than first breakpoint
%           - network_flag        network flag
%           - z                   current psc selection
%
% Output:   - Nref                number of reference points
%           - final_althyp_index  althyp_index used for unwrapped data
%           - network_flag        network flag
%
%
% ------------------------------------------------------------------------
% File............: ps_spatial_unwrapping.m
% Version & Date..: 1.7.2.16, 12-DEC-2009
% Author..........: Freek van Leijen
%                   Shizhuo Liu
%                   Delft Institute of Earth Observation and Space Systems
%                   Delft University of Technology
% ------------------------------------------------------------------------
%
% This software is developed by Delft University of Technology and is
% intended for scientific use only. Applications for commercial use are
% prohibited.
%
% Copyright (c) 2004-2009 Delft University of Technology, The Netherlands
%
% Change log
% v1.7.2.8, Shizhuo Liu
% - strong reduction in memory use
% - computation of test statistics in groups
% v1.7.2.8, Freek van Leijen
% - computation of test statistics only around removed arcs (speed up)
% v1.7.2.9, Freek van Leijen
% - flag for removed points, unwrap2_istep
% v1.7.2.12, Freek van Leijen
% - speed up and reduction of memory in computation of 'absolute' phase
%




% ------------------------------------------------------------------------
% Initialize
```

```
% --------------------------------------------------------------------

global Nlines Npixels az_spacing r_spacing fig Npar_max
global unwrap_istep unwrap2_istep project_id weighting
global ps_method visible_plots althyp_index althyp_index2 m2ph
global detrend_flag defo_model_flag results_id

fid_res = fopen([project_id '_resfile.txt'],'a');

fullscreen = [1 1 1280 1024];
%fullscreen=get(0,'Screensize');
[minratio,ratio_index] = min([fullscreen(3)/Npixels fullscreen(4)/Nlines]);
if ratio_index(1)==1 %(1) to prevent multiple maxima
  figpos = [1 1 fullscreen(3) fullscreen(3)*Nlines/Npixels];
elseif ratio_index(1)==2
  figpos = [1 1 fullscreen(4)*Npixels/Nlines fullscreen(4)];
end


% --------------------------------------------------------------------
% Initialize
% --------------------------------------------------------------------

final_althyp_index = [];
separate_flag = 0;
removed_points_flag=0;

if final_model<2
  error('The final model should be larger than ''1''.');
end


% --------------------------------------------------------------------
% Read data
% --------------------------------------------------------------------

dpsc_fid = fopen([project_id '_dpsc_sel' num2str(z) '.raw'],'r');
dpsc_data = fread(dpsc_fid,[2*Nifgs+2 Narcs(z)],'double')';
fclose(dpsc_fid);

dpsc_arcs = dpsc_data(:,1:2); % point indeces of the arcs
dpsc_phase = dpsc_data(:,3:Nifgs+2); % SD phase
dpsc_h2ph = dpsc_data(:,Nifgs+3:2*Nifgs+2); % mean h2ph of the arc
clear dpsc_data

dpsc_results_fid = fopen([project_id '_dpsc_results_sel' num2str(z)
'.raw'],'r');
dpsc_data = fread(dpsc_results_fid,[Nifgs+Npar_max+4 Narcs(z)],'double')';
fclose(dpsc_results_fid);

dpsc_param = dpsc_data(:,1:Npar_max); % estimated real parameters
dpsc_ens_coh = dpsc_data(:,Npar_max+1); % ensemble coherence
dpsc_acheck = dpsc_data(:,Npar_max+2:Npar_max+Nifgs+1); % estimated ambiguity
dpsc_model_info = dpsc_data(:,Npar_max+Nifgs+2:Npar_max+Nifgs+3); % model index
dpsc_varfac_best = dpsc_data(:,Npar_max+Nifgs+4); % posterior variance factor
clear dpsc_data

psc_fid = fopen([project_id '_psc_sel' num2str(z) '.raw'],'r');
psc_data = fread(psc_fid,[2*Nifgs+7 Npsc(z)],'double')';
fclose(psc_fid);

psc_array_in = psc_data(:,1:2);
psc_az = psc_data(:,5); % azimunt coordinates
psc_r = psc_data(:,6); % range coordinates
```

```
psc_h2ph = psc_data(:,7+Nifgs:6+2*Nifgs); % h2ph at pscs


switch ps_method
  case 'perio'

    % ---------------------------------------------------------------
    % Select the arcs with high coherence
    % ---------------------------------------------------------------

    low_coh = find(dpsc_ens_coh < ens_coh_threshold);

  case {'boot','ils'}

    % ---------------------------------------------------------------
    % Select the arcs with successful ambiguity resolution
    % ---------------------------------------------------------------

    low_coh = find(dpsc_varfac_best > varfac_threshold); % indices of pscs with
high variance

  otherwise

    error('Please select a valid ps method: ''period'', ''ils'' or ''boot''');

end

% delete those pscs which do not pass the OMT test in temporal unwrapping

dpsc_arcs_removed1 = dpsc_arcs(low_coh,:);
dpsc_arcs_orig = dpsc_arcs;

dpsc_phase(low_coh,:) = [];
dpsc_h2ph(low_coh,:) = [];
dpsc_arcs(low_coh,:) = [];
dpsc_param(low_coh,:) = [];
dpsc_ens_coh(low_coh) = [];
dpsc_acheck(low_coh,:) = [];
dpsc_varfac_best(low_coh) = [];
dpsc_model_info(low_coh,:) = [];
Narcs_new = size(dpsc_phase,1);

switch ps_method
 case 'perio'
  fprintf(fid_res,[datestr(now) ', unwrap_istep' num2str(unwrap_istep) ...
               ', unwrap2_istep' num2str(unwrap2_istep) ...
               ', number of apriori removed arcs in selection ' num2str(z) ':
' ...
               num2str(size(dpsc_arcs_removed1,1)) ' (coherence < ' ...
               num2str(ens_coh_threshold) ').\n']);
 case {'boot','ils'}
  fprintf(fid_res,[datestr(now) ', unwrap_istep' num2str(unwrap_istep) ...
               ', unwrap2_istep_' num2str(unwrap2_istep) ...
               ', number of apriori removed arcs in selection ' num2str(z) ':
' ...
               num2str(size(dpsc_arcs_removed1,1)) ' (variance factor > ' ...
               num2str(varfac_threshold) ').\n']);
end

% ---------------------------------------------------------------------
% Remove arcs which can not be tested (Nconnections <=2)
% ---------------------------------------------------------------------

exit_flag = 0;
```

```
dpsc_arcs_removed2 = [NaN NaN];

while exit_flag == 0
  psc = unique(dpsc_arcs); % unique psc points sorted in ascending order
  Npsc_connections = repmat(NaN,length(psc),1);
  for v = 1:length(psc)
    index_cns = find(dpsc_arcs(:)==psc(v)); % find indices of the current psc
                                            % in the arc list
    Npsc_connections(v) = length(index_cns); % the number of connection of
                                             % the current psc
  end

  single_psc = find(Npsc_connections<=2); % find pscs which has less than 2
connections (not testable)
  if ~isempty(single_psc)
    for v = 1:length(single_psc)
      [index_cns1,index_cns2] = find(dpsc_arcs==psc(single_psc(v)));% 2D
position of the (non-testable) pscs in the arc list

      dpsc_arcs_removed2 = [dpsc_arcs_removed2;dpsc_arcs(index_cns1,:)];

      dpsc_arcs(index_cns1,:) = [];
      dpsc_phase(index_cns1,:) = [];
      dpsc_h2ph(index_cns1,:) = [];
      dpsc_param(index_cns1,:) = [];
      dpsc_ens_coh(index_cns1) = [];
      dpsc_acheck(index_cns1,:) = [];
      dpsc_varfac_best(index_cns1) = [];
      Narcs_new = size(dpsc_phase,1); % update number of arcs

    end

  else
    exit_flag = 1;
  end

end

dpsc_arcs_removed2(1,:) = [];

switch ps_method
 case 'perio'
  fprintf(fid_res,[datestr(now) ', unwrap_istep' num2str(unwrap_istep) ...
               ', unwrap2_istep_' num2str(unwrap2_istep) ...
               ', number of untestable arcs in selection ' num2str(z) ': ' ...
               num2str(size(dpsc_arcs_removed2,1)) '.\n']);
 case {'boot','ils'}
  fprintf(fid_res,[datestr(now) ', unwrap_istep' num2str(unwrap_istep) ...
               ', unwrap2_istep_' num2str(unwrap2_istep) ...
               ', number of untestable arcs in selection ' num2str(z) ': ' ...
               num2str(size(dpsc_arcs_removed2,1)) '.\n']);
end

save([project_id '_apriori_removed_arcs_sel' num2str(z) '_' ...
      results_id '_unwrap_istep' num2str(unwrap_istep) ...
      '_unwrap2_istep' num2str(unwrap2_istep) '.mat'],...
     'psc_r','psc_az','dpsc_arcs_orig','dpsc_arcs_removed1',...
     'dpsc_arcs_removed2');

if Narcs_new==0 % no pscs are connected with more than 2 arcs
  error('There is no redundancy in the network');
end

% ---------------------------------------------------------------------
```

```
% Remove isolated points
% --------------------------------------------------------------------

psc = unique(dpsc_arcs); % the rest pscs after removing arcs
psc_az_new = psc_az(psc); % azimuth coordinates of the pscs
psc_r_new = psc_r(psc); % range coordinates of the pscs
isolated_psc = setdiff(1:Npsc(z),psc); % pscs which are not in the new arc list



% --------------------------------------------------------------------
% Set up connection vector
% --------------------------------------------------------------------

dpsc_arcs2 = reshape(dpsc_arcs(:,[1 2 2 1])',2,2*Narcs_new)';
connection_vec = zeros(Npsc(z),1,'int16');
cluster = 0;
while ~isempty(find(connection_vec==0))
  noconnection = find(connection_vec==0);
  if isempty(find(noconnection(1)==dpsc_arcs))
    connection_vec(noconnection(1)) = -1;
  else
    cluster = cluster+1;
    connection_vec(noconnection(1)) = cluster;
    cluster_array = connection_vec(dpsc_arcs2)==cluster;
    cluster_index = find(cluster_array(:,1) & ~cluster_array(:,2));
    while ~isempty(cluster_index)
      connection_vec(dpsc_arcs2(cluster_index,2)) = cluster;
      cluster_array = connection_vec(dpsc_arcs2)==cluster;
      cluster_index = find(cluster_array(:,1) & ~cluster_array(:,2));
    end
  end
end


% --------------------------------------------------------------------
% Selection of the reference point
% --------------------------------------------------------------------

central_az = Nlines*az_spacing/2; % centre coordinates
central_r = Npixels*r_spacing/2;
% Nref is initialized as Nan

if Nref(z)>=1 %use existing reference point
  ref_fid = fopen([project_id '_ref_sel' num2str(z) '.raw'],'r');
  ref_array = fread(ref_fid,[3 Nref(z)],'double')';
  fclose(ref_fid);

  ref1_az = ref_array(1,2);
  ref1_r = ref_array(1,3);
  index = find(psc_az_new == ref1_az); % the new index in the updated psc
(connection >2)
  index2 = find(psc_r_new(index) == ref1_r); % the new index in range
  index = index(index2); % the new index both for azimuth and range

  if ~isempty(index)
    ref_array = [psc(index) ref1_az ref1_r]; % index and coordinates
  else % if the previous reference had been rejected, then find a new one
    switch ps_method
      case 'perio'
        [dummy,index] = max(dpsc_ens_coh); % select arcs with maximal coherence
        index = dpsc_arcs(index,1); % and choose one of the two nodes
        ref_array = [index psc_az(index) psc_r(index)];
```

```
      case {'boot','ils'}
        [dummy,index] = min(dpsc_varfac_best); % select arcs with maximal
coherence
        index = dpsc_arcs(index,1); % and choose one of the two nodes
        ref_array = [index psc_az(index) psc_r(index)];

      otherwise
        error('Please select a valid ps method: ''perio'', ''ils'' or ''boot');

    end
  end

else % determine new reference point
     % new reference has lowest posteriori variance factor
  if ~isempty(ref_cn) % use the one pre-specified
    index = find(psc_az_new == ref_cn(1,1));
    index2 = find(psc_r_new(index) == ref_cn(1,2));
    index = index(index2);
    if isempty(index)
      error('Specified reference point is not a PS');
    else
      ref_array = [psc(index) ref_cn];
    end

  else

    switch ps_method
      case 'perio'
        [dummy,index] = max(dpsc_ens_coh); % select arcs with maximal coherence
        index = dpsc_arcs(index,1); % and choose one of the two nodes
        ref_array = [index psc_az(index) psc_r(index)];

      case {'boot','ils'}
        [dummy,index] = min(dpsc_varfac_best); % select arcs with maximal
coherence
        index = dpsc_arcs(index,1); % and choose one of the two nodes
        ref_array = [index psc_az(index) psc_r(index)];

      otherwise
        error('Please select a valid ps method: ''perio'', ''ils'' or ''boot');

    end
  end
end

Nref(z) = 1; % only one in each iteration


% ----------------------------------------------------------------------
% Determine the pseudo-reference points (in case of separate networks)
% ----------------------------------------------------------------------
% the following code does: find pscs which have more than 2 connections
% (i.e. within a certain network) but not connected with the reference
% point. In that case, use the one of the points (cluster) which is
% closest to the centre point of the full crop

psc_temp = psc;
psc_array = [(1:Npsc(z))' zeros(Npsc(z),1)];
count = 0;
while ~isempty(psc_temp)
  count = count + 1;

  if count >= 2 % initialize it with the first isolated psc
    ref_array(count,:) = [psc_temp(1) NaN NaN]; % random chosen reference points
```

```
                                                % (first reference point was
already selected)
  end
  % cluster is a vector contains row indices with identical column (index of the
previous reference point)
  %%%cluster = find(link(:,ref_array(count,1)) == true); % all pscs connected
with the reference point
  cluster = find(connection_vec==count); % all pscs connected with the reference
point
  % mark the pscs which is connected with the previous reference point
  psc_array(cluster,2) = count; % to specify the groups
                                % count = 0 are the isolated points
  % pscs which have more than 2 connections but not connected with the reference
point
  psc_temp = setdiff(psc_temp,cluster);

  if count >= 2
    dist = sqrt((psc_az(cluster)*az_spacing-central_az).^2 +
(psc_r(cluster)*r_spacing-central_r).^2);
    [dummy,index] = min(dist);
    ref_array(count,:) = [cluster(index) psc_az(cluster(index))
psc_r(cluster(index))]; % select most central psc

% to improve kriging estimate
  end

end

Nref(z) = size(ref_array,1); % update number of reference point
non_ref_points = setdiff(psc,ref_array(:,1)); % number of points which are not
chosen as reference point



if Nref(z)>1 & unwrap_istep<5 & sum(abs(psc_array_in(:,2)-psc_array(:,2)))~=0
  % last conditions checks whether anything changed compared to the
  % last loop. If not, it does not make sense to do another.

  % ---------------------------------------------------------------------
  % If separate networks, save psc to file and form a new network
  % ---------------------------------------------------------------------

  psc_data = [psc_array psc_data(:,3:end)];
  psc_fid = fopen([project_id '_psc_sel' num2str(z) '.raw'],'w');
  fwrite(psc_fid,psc_data','double');
  fclose(psc_fid);

  ref_fid = fopen([project_id '_ref_sel' num2str(z) '.raw'],'w');
  fwrite(ref_fid,ref_array','double');
  fclose(ref_fid);

else

  % ---------------------------------------------------------------------
  % Testing and unwrapping of the ambiguities
  % ---------------------------------------------------------------------

  psc_phase_unw = repmat(NaN,Npsc(z),Nifgs);
  psc_acheck_temp = repmat(NaN,Npsc(z),Nifgs);

  max_con = 0;
  for v = 1:Npsc % number of connections for each psc, for isolated psc the
number is 0
    max_con = max(max_con,length(find(dpsc_arcs(:)==v)));
```

```
end

a0 = 0.1;
g0 = 0.5;
kb = repmat(NaN,max_con,1);
ab = repmat(NaN,max_con,1);

for v = 1:max_con
    % lam0 : non-centre parameter of chi-square distribution
    % k1: critical value for 1d test
    % kb: critical value of b-dimensional test
    % level of significance for b-dimensional test
  [lam0,k1,kb(v),ab(v)] = fvl_pretest(v,a0,g0);
end

Narcs_new_orig = Narcs_new;
orig_index = (1:Narcs_new)';
y = dpsc_acheck; % estimated ambiguity

A = sparse(Narcs_new,Npsc(z));
for v = 1:Narcs_new
  A(v,dpsc_arcs(v,1)) = -1;
  A(v,dpsc_arcs(v,2)) = 1;
end
% ignore isolated psc as well as reference point
% remove corresponding columns
% refer to Eq.4.11 in BK's thesis
A(:,[isolated_psc ref_array(:,1)']) = [];

kOMT = 1e-10;
adapt_vec = [NaN NaN]; % needed to start with
adapt_vec_new = [];
exit_flag = 0;
counter = 0;

while exit_flag == 0

  Qyy = speye(Narcs_new);
  N = A'*A;% size: Npsc*Npsc
  cond_N = condest(N);

  if cond_N>1e5 % separate networks!

    if unwrap_istep<5 & sum(abs(psc_array_in(psc_array_in(:,2)==0,2)-...
                        psc_array(psc_array_in(:,2)==0,2)))~=0
    % last conditions checks whether there are any new isolated
      % points. If not, it does not make sense to do another loop.

    separate_flag = 1;
    exit_flag = 1;
    else

    % -----------------------------------------------------------------
    % Set up connection vector
    % -----------------------------------------------------------------

      dpsc_arcs2 = reshape(dpsc_arcs(orig_index,[1 2 2 1])',2,2*Narcs_new)';
      connection_vec = zeros(Npsc(z),1,'int16');
      cluster = 0;
      while ~isempty(find(connection_vec==0))
        noconnection = find(connection_vec==0);
        if isempty(find(noconnection(1)==dpsc_arcs(orig_index,:)))
          connection_vec(noconnection(1)) = -1;
        else
```

```
            cluster = cluster+1;
            connection_vec(noconnection(1)) = cluster;
            cluster_array = connection_vec(dpsc_arcs2)==cluster;
            cluster_index = find(cluster_array(:,1) & ~cluster_array(:,2));
            while ~isempty(cluster_index)
              connection_vec(dpsc_arcs2(cluster_index,2)) = cluster;
              cluster_array = connection_vec(dpsc_arcs2)==cluster;
              cluster_index = find(cluster_array(:,1) & ~cluster_array(:,2));
            end
          end
        end

      psc_temp = unique(dpsc_arcs(orig_index,:));

      count = 0;
      while ~isempty(psc_temp)
        count = count + 1;

        if count > Nref(z)
          ref_array(count,:) = [psc_temp(1) NaN NaN]; % random chosen reference
points
                                          % (first reference point was already
selected)
        end
        cluster = find(connection_vec==count);

        psc_array(cluster,2) = count; % to specify the groups
                              % count = 0 are the
                                    % isolated points
          psc_temp = setdiff(psc_temp,cluster);

        if count > Nref(z)
          dist = sqrt((psc_az(cluster)*az_spacing-central_az).^2 +
(psc_r(cluster)*r_spacing-central_r).^2);
          [dummy,index] = min(dist);
          ref_array(count,:) = [cluster(index) psc_az(cluster(index)) ...
                psc_r(cluster(index))];
          % select most central psc
          % to improve kriging estimate
          Nref(z) = Nref(z)+1;
          ref_index = find(non_ref_points==ref_array(Nref(z),1));
          A(:,ref_index) = [];
          non_ref_points(ref_index) = [];
          index = find(sum(abs(A),2)==0);
          if ~isempty(index)
            A(index,:) = [];
            y(index,:) = [];
            orig_index(index) = [];
            Narcs = size(y,1);
            Qyy = speye(Narcs); %sparse eye
          end
        end %end if count > Nref(z)
      end %end while ~isempty(psc_temp)
      N = A'*A;
      end %end if unwrap_istep<5
    end %end if cond_N>1e5 % separate networks!

    if exit_flag==0
      R = chol(N);

      rhs = A'*y;
      acheck = R\(R'\rhs);

      ycheck  = A*acheck;
```

```
      echeck  = y-ycheck;

      Npsc_estimable = size(A,2);

      OMT = diag(echeck'*echeck);

      if sum(OMT) > kOMT

      Rinv  = R\eye(Npsc_estimable);
      %Rinv  = full(R)\eye(Npsc_estimable,'single');
      % faster and lower memory use (single precision)

      if rem(counter,20)==0
        %first and every 20th time (re-) compute full

        Qecheck_diag = repmat(NaN,Narcs_new,1);
        TTq = repmat(NaN,size(non_ref_points));

        %H0    = A*Rinv;
        %Qycheck = H0*H0';
        %Qecheck = Qyy-Qycheck;

        [Ngroup, Np_group, arc_group, N_arc_avg] =
ps_spatial_unwrapping_grouping(A);

        if Ngroup>1

          save Rinv Rinv   % save it and reload it from file if needs
                      % size of it is Npsc * Npsc, can be quite big
                      % if Npsc it large.

            fprintf(1, 'Grouping finished \n') ;
            fprintf(1,'The Network is grouped into %4.0f groups of %4.0f arcs
and %4.0f points \n',Ngroup, N_arc_avg, Np_group);

        end

        n = 1;
        sing_exit = 0; %exit flag for singular network
        while n<=Ngroup & sing_exit==0

          if Ngroup==1 % no grouping
            arc_list = arc_group{1};
            Rinv = A*Rinv;
          else
            load Rinv % Rinv is changed, so reload it, only reload
                 % Ngroup times for each while iteration

              arc_list = arc_group{n};
              Rinv = A(arc_list,:)*Rinv; % use same name to save memory
          end

          % compute diagonal of Qecheck here
          Qecheck_diag(arc_list,1) = 1 - sum(Rinv.^2,2);

          sing_index = find(Qecheck_diag<=1e-8);

          if isempty(sing_index) % new since 21-08-09, to account for
                          % single connection between networks

            if n<Ngroup
            v_end = n*Np_group;
            else
            v_end = size(non_ref_points,1);
```

```
      end

      for v = (n-1)*Np_group+1:v_end
      arc_index = find(A(:,v)~=0);
      arc_index(1)=[]; % to create basis, see e.g. verhoef97
      Narc_index = length(arc_index);
      %Cq = zeros(Narcs_new,Narc_index);
      %for vv = 1:Narc_index
      %Cq(arc_index(vv),vv) = 1;
      %end
      %temp1 = repmat(NaN,Narc_index,Nifgs);
      %temp2 = repmat(NaN,Narc_index,Narc_index);
      temp1 = echeck(arc_index,:);

      [dummy,dummy,arc_index2] = intersect(arc_index,arc_list);
      % compute part of Qecheck
      % temp2 is part of Qecheck
      temp2 = eye(Narc_index) - Rinv(arc_index2,:)*Rinv(arc_index2,:)';
      %->Qecheck = Qy-Qycheck

      % teststatistics
      if condest(temp2)>1e5 % new since 21-08-09, to account for
                        % single point between networks
            TTq(v) = 0;
      else
        Tq = sum(diag((temp1'/temp2)*temp1));
        %Tq(v) = sum(diag(echeck'*Cq*inv(Cq'*Qecheck*Cq)*Cq'*echeck));
        TTq(v) = Tq/kb(Narc_index);
      end
      clear temp1 temp2
      end %end for Np_group

    else
      sing_exit = 1;
    end %end if isempty(sing_index)

    n = n+1;

  end %end while n<=Ngroup & sing_exit==0

else %only correct values in neighborhood of removed arc(s)

  Nadapt = size(adapt_vec_new,1);
  psc_adapt = unique(reshape(dpsc_arcs(adapt_vec_new(:,1),:),2*Nadapt,1));
  dpsc_arcs_vec = reshape(dpsc_arcs(orig_index,:),2*Narcs_new,1);

  sing_exit = 0; %exit flag for singular network
  nodes_done = [];

  %%%%%%%%%%%%%%%%
  %while loop turned off, only directly neighboring points
    %seems significant, FvL 15-10-09
  %%%%%%%%%%%%%%%%%%%
  %%%TTq_max_diff = inf;
  %while  TTq_max_diff>10 & sing_exit==0
  %www=0;
  %while www<2 & sing_exit==0
  %www=www+1
    arc_index = find(ismember(dpsc_arcs_vec,psc_adapt));
    arc_index = unique(mod(arc_index-1,Narcs_new)+1);%connecting arcs

    arcs = dpsc_arcs(orig_index(arc_index),:);
    nodes = unique(arcs(:));
    nodes = setdiff(nodes,nodes_done);%remove nodes already done
```

```
          [dummy,dummy,node_list] = intersect(nodes,non_ref_points);
          Nnodes = size(node_list,1);
          [arc_list,dummy] = find(A(:,node_list)~=0);
          arc_list = unique(arc_list);

          Rinv_new = A(arc_list,:)*Rinv; % use same name to save memory
          Qecheck_diag(arc_list,1) = 1 - sum(Rinv_new.^2,2);

          sing_index = find(Qecheck_diag<=1e-8);

          if isempty(sing_index) % new since 21-08-09, to account for
                          % single connection between networks

            for v = 1:Nnodes
            arc_index = find(A(:,node_list(v))~=0);
            arc_index(1)=[]; % to create basis, see e.g. verhoef97
            Narc_index = length(arc_index);
            temp1 = echeck(arc_index,:);
            [dummy,dummy,arc_index2] = intersect(arc_index,arc_list);
            temp2 = eye(Narc_index) -
Rinv_new(arc_index2,:)*Rinv_new(arc_index2,:)';
            if condest(temp2)>1e5 % new since 21-08-09, to account for
                            % single point between networks
                  TTq(node_list(v)) = 0;
            else
              Tq = sum(diag((temp1'/temp2)*temp1));
              TTq(node_list(v)) = Tq/kb(Narc_index);
            end
            clear temp1 temp2
            end %end for v = 1:Nnodes

            nodes_done = [nodes_done; psc_adapt];
            psc_adapt = nodes;
            %%%TTq_max_diff = max(abs(TTq(node_list)-TTq_old(node_list)));
          else
            sing_exit = 1;
          end %end if isempty(sing_index)

          %%%%%%%%%%%%%%%
          %while loop switched off
          %%%%%%%%%%%%%%%
          %end %end while
          %end %end while www<2 & sing_exit==0

      end %end if rem(counter,20)==0

      w = wtest_2d(Qecheck_diag,echeck,Nifgs) ; % only the diagonal of Qecheck
is needed

      clear Rinv % no need anymore, release memory

        if sing_exit==0
        TT1 = sum(w.^2,2)/(k1^2);
        [T1max,indexw] = max(TT1);
        T1max = T1max(1); % to avoid multiple maxima
        indexw = indexw(1); % to avoid multiple maxima
        [Tbmax,indexb] = max(TTq);
        Tbmax = Tbmax(1); % to avoid multiple maxima
        indexb = indexb(1); % to avoid multiple maxima

        if T1max>1
          if T1max>Tbmax
            test_result = 1;
```

```
          else
            test_result = 0;
          end

          adapt_vec_new = [];

          if test_result == 0
            arc_index = find(A(:,indexb)~=0);
            y(arc_index,:) = [];
            A(arc_index,:) = [];
            A(:,indexb) = [];
            adapt_vec_new = [adapt_vec_new; [orig_index(arc_index)
repmat(NaN,size(arc_index,1),1)]];
            adapt_vec = [adapt_vec; [orig_index(arc_index)
repmat(NaN,size(arc_index,1),1)]];
            orig_index(arc_index) = [];
            psc_array(non_ref_points(indexb),2) = 0;
            non_ref_points(indexb) = [];
            TTq(indexb) = [];%%%
            TT1(arc_index) = [];%%%
            Qecheck_diag(arc_index) = [];%%%
            Narcs_new = Narcs_new - size(arc_index,1);

          elseif test_result == 1
            y(indexw,:) = [];
            A(indexw,:) = [];
            adapt_vec_new = [adapt_vec_new; [orig_index(indexw) NaN]];
            adapt_vec = [adapt_vec; [orig_index(indexw) NaN]];
            orig_index(indexw) = [];
            TT1(indexw) = [];%%%
            Qecheck_diag(indexw) = [];%%%
            Narcs_new = Narcs_new - 1;

          end %end if test_result == 0

          psc_iso1 = find(sum(abs(A),1)==1);
          while ~isempty(psc_iso1)
            Niso = length(psc_iso1);
            dpsc_iso = repmat(NaN,Niso,1);
            for h = 1:Niso
            dpsc_iso(h) = find(A(:,psc_iso1(h))~=0);
            end
            dpsc_iso = unique(dpsc_iso);
            Ndiso = length(dpsc_iso);
            A(dpsc_iso,:) = [];
            y(dpsc_iso,:) = [];
            A(:,psc_iso1) = [];
            psc_array(non_ref_points(psc_iso1),2) = zeros(Niso,1);
            non_ref_points(psc_iso1) = [];
            TTq(psc_iso1) = [];%%%
            TT1(dpsc_iso) = [];%%%
            Qecheck_diag(dpsc_iso) = [];%%%
            adapt_vec_new = [adapt_vec_new; [orig_index(dpsc_iso)
repmat(NaN,Ndiso,1)]];
            adapt_vec = [adapt_vec; [orig_index(dpsc_iso) repmat(NaN,Ndiso,1)]];
            orig_index(dpsc_iso) = [];
            Narcs_new = Narcs_new - Ndiso;

            psc_iso1 = find(sum(abs(A),1)==1);
          end

          psc_iso2 = find(sum(abs(A),1)==0);
          if ~isempty(psc_iso2)
            Niso = length(psc_iso2);
```

```
            A(:,psc_iso2) = [];
            psc_array(non_ref_points(psc_iso2),2) = zeros(Niso,1);
            non_ref_points(psc_iso2) = [];
            TTq(psc_iso2) = [];%%%
          end

        else
          exit_flag = 1; %2nd if
        end %end if T1max>1

      else % singular network (only one connection)
        y(sing_index,:) = [];
        A(sing_index,:) = [];
        adapt_vec_new = [adapt_vec_new; [orig_index(sing_index) ...
                repmat(NaN,length(sing_index),1)]];
        adapt_vec = [adapt_vec; [orig_index(sing_index) ...
                repmat(NaN,length(sing_index),1)]];
        orig_index(sing_index) = [];
        TT1(sing_index) = [];%%%
        Qecheck_diag(sing_index) = [];%%%
        Narcs_new = Narcs_new - length(sing_index);

        fprintf(fid_res,[datestr(now) ', removed arc from network causing
separate networks.\n']);

      end %end if sing_exit==0

      else
      exit_flag = 1;
      end %end if sum(OMT) > kOMT

      if Narcs_new==0
      exit_flag=1;
      end

      counter = counter + 1;
      if counter == Narcs_new_orig*Nifgs
      exit_flag = 1;
      %          error('The unwrapping of ifgs %4.0f did not converge to a
solution',v);
      % in the futur something can be done here, e.g., remove
      % this ifgs from the set (adapt Btemp, Nifgs, ....)
      end
    end %end if exit_flag==0

    %TTq_old = TTq;
    %TT1_old = TT1;
    %Qecheck_diag_old = Qecheck_diag;

  end %end while exit_flag == 0

  adapt_vec(1,:) = []; % remove NaN

  save([project_id '_removed_arcs_sel' num2str(z) '_' results_id ...
      '_unwrap_istep' num2str(unwrap_istep) ...
      '_unwrap2_istep' num2str(unwrap2_istep) '.mat'],'psc_r','psc_az',...
       'psc_az_new','psc_r_new','dpsc_arcs','ref_array','adapt_vec');

  fprintf(fid_res,[datestr(now) ', unwrap_istep' num2str(unwrap_istep) ...
              ', unwrap2_istep' num2str(unwrap2_istep) ...
           ', number of removed arcs in selection ' num2str(z) ': ' ...
           num2str(size(adapt_vec,1)) ', which is ' ...
           num2str(100*size(adapt_vec,1)/Narcs_new_orig,'%5.2f') ...
           ' percent of the total.\n']);
```

```
  fprintf(1,[datestr(now) ', unwrap_istep' num2str(unwrap_istep) ...
              ', unwrap2_istep' num2str(unwrap2_istep) ...
           ', number of removed arcs in selection ' num2str(z) ': ' ...
           num2str(size(adapt_vec,1)) ', which is ' ...
           num2str(100*size(adapt_vec,1)/Narcs_new_orig,'%5.2f') ...
           ' percent of the total.\n']);

  if Narcs_new==0
    error('All arcs have been rejected. Please change your settings and try
again');
  end


  if separate_flag==1

    % ------------------------------------------------------------------
    % If separate networks, save psc to file and form a new network
    % ------------------------------------------------------------------

    psc_data = [psc_array psc_data(:,3:end)];
    psc_fid = fopen([project_id '_psc_sel' num2str(z) '.raw'],'w');
    fwrite(psc_fid,psc_data','double');
    fclose(psc_fid);

    ref_fid = fopen([project_id '_ref_sel' num2str(z) '.raw'],'w');
    fwrite(ref_fid,ref_array','double');
    fclose(ref_fid);

  else

    % ------------------------------------------------------------------------
    % Fixing of individual unwrapping errors
    % ------------------------------------------------------------------------

    adapt_vec = [NaN NaN]; % needed to start with
    dpsc_acheck_new = dpsc_acheck(orig_index,:); % for fixing algorithm
    dpsc_arcs_new = dpsc_arcs(orig_index,:);
    echeck = repmat(NaN,size(dpsc_acheck_new));

    for v = 1:Nifgs

      exit_flag = 0;
      skip_flag = 0;
      orig_xcheck_flag = 0;
      counter = 0;
      index_vec = [NaN NaN];
      y = dpsc_acheck_new(:,v);

      while exit_flag == 0 & counter<Narcs_new

      rhs = A'*y;
      xcheck = R\(R'\rhs);
      ycheck = A*xcheck;
      echeck_ifgs = y-ycheck;

      OMT = echeck_ifgs'*echeck_ifgs;
      if sum(OMT) > 1e-10
        if skip_flag == 0
          [dummy,index1] = max(abs(echeck_ifgs));
        else
          [sort_echeck,sort_index] = sort(abs(echeck_ifgs));
          index1 = sort_index(end-skip_flag);
        end
```

```
        if round(abs(echeck_ifgs(index1(1))))>=1;
          y(index1(1)) = y(index1(1))-round(echeck_ifgs(index1(1)));
        elseif echeck_ifgs(index1(1))>0
          y(index1(1)) = y(index1(1))-1;
        elseif echeck_ifgs(index1(1))<0
          y(index1(1)) = y(index1(1))+1;
        else
          error('This is not possible...');
        end

        index_old = index_vec;
        index_vec = [index1(1) v];

        if index_vec == index_old % to avoid 1 -1 1 .. of same arc
          skip_flag=skip_flag+1;
          %adapt_vec(end,:) = [];
        else
          skip_flag=0;
          % register number of adaptions
          if isempty(find(ismember(adapt_vec,index_vec,'rows')));
            adapt_vec = [adapt_vec; index_vec];
          end
        end

      else
        exit_flag = 1;
      end

      counter = counter + 1;

      end %while
      dpsc_acheck_new(:,v) = y;
      acheck(:,v) = xcheck;
    end

    adapt_vec(1,:) = []; % remove NaN

    save([project_id '_adapted_arcs_sel' num2str(z) '_' results_id ...
          '_unwrap_istep' num2str(unwrap_istep) ...
        '_unwrap2_istep' num2str(unwrap2_istep) ...
        '.mat'],'psc_r','psc_az',...
         'dpsc_arcs_new','ref_array','adapt_vec');

    fprintf(fid_res,[datestr(now) ', unwrap_istep' num2str(unwrap_istep) ...
               ', unwrap2_istep' num2str(unwrap2_istep) ...
                     ', number of adapted ambiguities in selection ' ...
                     num2str(z) ': ' num2str(size(adapt_vec,1)) ', which is
' ...
                     num2str(100*size(adapt_vec,1)/(Narcs_new_orig*Nifgs), ...
                             '%5.2f') ' percent of the total.\n']);

    fprintf(1,[datestr(now) ', unwrap_istep' num2str(unwrap_istep) ...
               ', unwrap2_istep' num2str(unwrap2_istep) ...
               ', number of adapted ambiguities in selection ' ...
               num2str(z) ': ' num2str(size(adapt_vec,1)) ', which is ' ...
               num2str(100*size(adapt_vec,1)/(Narcs_new_orig*Nifgs), ...
                       '%5.2f') ' percent of the total.\n']);


    % --------------------------------------------------------------------
    % Calculating unwrapped phase
    % --------------------------------------------------------------------
```

```
    psc_acheck_temp(non_ref_points,:) = round(acheck); % rounding to ensure
integers

    rm_index = [];
    for v = 1:Nref(z)
      indexx = find(psc_array(:,2)==v); % find the pscs which belong to the
current reference point group
      if length(indexx)==1
      psc_array(indexx,2) = 0; % single point in the network
      rm_index = [rm_index v]; % remove it from the list
      end
    end
    ref_array(rm_index',:) = [];
    Nref(z) = Nref(z)-size(rm_index,2); % update number of reference points
    indexx = find(psc_array(:,2)~=0);
    clusters = unique(psc_array(indexx,2)); % indentity number of sub-networks
    if Nref(z)~=length(clusters)
      error('Something went wrong in the removal of clusters');
    else
      for v = 1:Nref(z)
      indexx2 = find(psc_array(indexx,2)==clusters(v)); % index of pscs
belonging to a certain sub-network
      psc_array(indexx(indexx2),2) = v;
      indexx(indexx2) = [];
      end
    end


    psc_phase =  R\(R'\(A'*dpsc_phase(orig_index,:)));
    psc_phase_unw(non_ref_points,:) = 2*pi*psc_acheck_temp(non_ref_points,:)
+psc_phase;

    if Nref(z)>1 & unwrap_istep<5 &
sum(abs(psc_array_in(psc_array_in(:,2)==0,2)-...
                        psc_array(psc_array_in(:,2)==0,2)))~=0
      % last conditions checks whether there are any new isolated
        % points. If not, it does not make sense to do another loop.


      % ------------------------------------------------------------------
      % If separate networks, save psc to file and form a new network
      % ------------------------------------------------------------------

      psc_data = [psc_array psc_data(:,3:end)];
      psc_fid = fopen([project_id '_psc_sel' num2str(z) '.raw'],'w');
      fwrite(psc_fid,psc_data','double');
      fclose(psc_fid);

      ref_fid = fopen([project_id '_ref_sel' num2str(z) '.raw'],'w');
      fwrite(ref_fid,ref_array','double');
      fclose(ref_fid);

    else

      % ------------------------------------------------------------------
      % Add the modeled deformation (if needed)
      % ------------------------------------------------------------------

      switch defo_model_flag
       case 'yes'
        psc_defo_fid = fopen([project_id '_psc_defo_sel' num2str(z)
'.raw'],'r+');
        psc_defo = fread(psc_defo_fid,[Nifgs Npsc(z)],'double')';
        psc_phase_unw_tot = psc_phase_unw+psc_defo;
```

```
       otherwise
        psc_phase_unw_tot = psc_phase_unw;
      end

      % ------------------------------------------------------------
      % Calculation of the parameters of interest
      % ------------------------------------------------------------
      % Here we can introduce a stochastic model, assuming that the
      % ambiguities are deterministic, hence, a success rate of 1.

      psc_param = repmat(NaN,Npsc(z),Npar_max);
      psc_covar = repmat(NaN,Npsc(z),Npar_max*(Npar_max+1)/2);
      psc_ens_coh = repmat(NaN,Npsc(z),1);
      psc_phase_res = repmat(NaN,Npsc(z),Nifgs);
      psc_defo = repmat(NaN,Npsc(z),Nifgs);
      psc_acheck = repmat(NaN,Npsc(z),Nifgs);
      psc_sig2hat = repmat(NaN,Npsc(z),1);


      % ------------------------------------------------------------
      % Construct stochastic model
      % ------------------------------------------------------------

      if isempty(sig2_est)
      if final_model == 1 % master atmosphere stochastic
        sig2 = [(pi*15/180)^2; repmat((pi*20/180)^2,Nifgs,1)];
      else % master atmosphere estimated
        sig2 = [0; repmat((pi*30/180)^2,Nifgs,1)];
      end
      else
      sig2 = sig2_est;
      end

      Qy = repmat(2*sig2(1),Nifgs,Nifgs);
      for v = 1:Nifgs
      Qy(v,v) = Qy(v,v)+2*sig2(v+1);
      end
      invQy = inv(Qy);


      % ------------------------------------------------------------
      % Select desired model
      % ------------------------------------------------------------

      [Npar,par_index,covar_index] = ps_model_definitions('Npar',final_model);
      [psc_annotation,par_index,covar_index] =
ps_model_definitions('annotation',final_model);
      redun = Nifgs-Npar; %redundancy

      if isempty(breakpoint)
      althyp_index = median(dpsc_model_info(~isnan(dpsc_model_info(:,2)),2));
      else
      althyp_index = breakpoint;
      end

      final_althyp_index = althyp_index; % for parsing to ps_est.m
                               %choose most common althyp_index (only used by
some models)

       for v = non_ref_points'

       h2ph = psc_h2ph(v,:)';
       [B1Qy2,par_index,covar_index,defo_index] =
ps_model_definitions('model',final_model,Nifgs,h2ph,Btemp,Bdop,std_param);
```

```
althyp_index = final_althyp_index; % reset althyp_index
B1 = B1Qy2(1:Nifgs,:);

switch weighting
 case 'unw'
  N = B1'*B1;
  R = chol(N);
  rhs =  R\(R'\B1');
 case {'weight','vce'}
  N = B1'*invQy*B1;
  R = chol(N);
  rhs =  R\(R'\(B1'*invQy));
end

y = psc_phase_unw(v,:)';
xhat = rhs*y;
yhat = B1*xhat;
ehat = y-yhat;

y2 = psc_phase_unw_tot(v,:)';
xhat2 = rhs*y2;
yhat2 = B1*xhat2;
ehat2 = y2-yhat2;

switch weighting
 case 'vce'

  %% //gkstart
  %% ----------------------------
  %% VCE
  %% ----------------------------
  %VCE=vcem3(A,y,Q_cofactor);
  %Qy_new=VCE(1)*Q_cofactor(:,:,1)+VCE(2)*Q_cofactor(:,:,2);
  %
  %% check for positive definiteness
  %if and(sum(find(eig(Qy_new)<=0))==0, sum(find(VCE<=0))==0)
  %  invQy = inv(Qy_new); % positive definite
  %else
  %  % not positive definite; calculate 1 variance factor instead
  %  Qy_new=((ehat'*invQy*ehat)/b)*sum(Q_cofactor,3); % sig2hat*Qy
  %  invQy = inv(Qy_new);
  %end
  %% //gkend

  Tb = ehat'*invQy*ehat; % overall model test
  sig2hat = Tb/redun;        % estimate variance factor

  Qy_new=sig2hat*Qy;
  invQy_new = inv(Qy_new);

  N = B1'*invQy_new*B1;
  R = chol(N);
  rhs = R\(R'\(B1'*invQy_new));
  xhat = rhs*y;
  yhat = B1*xhat;
  ehat = y-yhat;
  Rinv  = R\eye(size(B1,2));
  Qahat = Rinv*Rinv';

  covar_reshape = reshape(triu(Qahat),1,Npar^2);
  psc_covar(v,covar_index) = covar_reshape(covar_reshape~=0);
  psc_sig2hat(v) = sig2hat;

end
```

```
        psc_param(v,par_index) = xhat2';
        psc_phase_res(v,:) = ehat2';
        psc_defo(v,:) = (B1(:,defo_index)*xhat2(defo_index)+ehat2)'/m2ph;
        psc_ens_coh(v) = abs((1/Nifgs)*sum(exp(i*ehat')));
        psc_acheck(v,:) = psc_acheck_temp(v,:);
        end

        psc_param(ref_array(:,1),par_index) = zeros(Nref(z),Npar);
        psc_covar(ref_array(:,1),covar_index) = zeros(Nref(z),Npar*(Npar+1)/2);
        psc_phase_res(ref_array(:,1),:) = zeros(Nref(z),Nifgs);
        psc_defo(ref_array(:,1),:) = zeros(Nref(z),Nifgs);
        psc_ens_coh(ref_array(:,1)) = ones(Nref(z),1);
        psc_acheck(ref_array(:,1),:) = zeros(Nref(z),Nifgs);




        % ------------------------------------------------------------------
        % Plot histograms
        % ------------------------------------------------------------------

        % switched off, because was causing unexplainable errors on
        % the cluster, FvL 15-10-09

        %if length(non_ref_points)>=100

        %for v = 1:Npar
        %    fig = fig+1;
        %    figure(fig);
        %    if strcmp(visible_plots,'n')
        %         set(gcf,'visible','off');
        %    end
        %
hist(psc_param(non_ref_points,par_index(v)),length(non_ref_points)/10);
        %    xlabel(['Model: ' num2str(final_model) ', parameter: '
char(psc_annotation(par_index(v)))],'interpreter','none');
        %    print('-dpng',['plots/' project_id '_hist_psc_parameter_'
char(psc_annotation(par_index(v))) '_model' num2str(final_model) '_sel'
num2str(z) '_' results_id '.png']);
        %  end

        % fig = fig+1;
        %     figure(fig);
        % if strcmp(visible_plots,'n')
        %       set(gcf,'visible','off');
        %  end
        %     hist(psc_ens_coh(non_ref_points),length(non_ref_points)/10);
        %     xlabel('psc_ens_coh','interpreter','none')
        %     print('-dpng',['plots/' project_id
'_hist_psc_parameter_ens_coh_model' num2str(final_model) '_sel' num2str(z) '_'
results_id '.png']);

        %       end



        % ------------------------------------------------------------------
        % Write results to file
        % ------------------------------------------------------------------

        psc_total = [psc_param psc_ens_coh psc_sig2hat ...
                 psc_phase_res psc_acheck psc_defo];
        psc_fid = fopen([project_id '_psc_results_sel' num2str(z) '.raw'],'w');
```

```
       fwrite(psc_fid,psc_total','double');
       fclose(psc_fid);

       switch weighting
      case 'vce'
       psc_fid = fopen([project_id '_psc_results_covar_sel' num2str(z)
'.raw'],'w');
       fwrite(psc_fid,psc_covar','double');
       fclose(psc_fid);
       end

       psc_data = [psc_array psc_data(:,3:end)];
       psc_fid = fopen([project_id '_psc_sel' num2str(z) '.raw'],'w');
       fwrite(psc_fid,psc_data','double');
       fclose(psc_fid);

       ref_fid = fopen([project_id '_ref_sel' num2str(z) '.raw'],'w');
       fwrite(ref_fid,ref_array','double');
       fclose(ref_fid);

       network_flag = 1;

       if sum(abs(psc_array_in(:,2)-psc_array(:,2)))~=0
       removed_points_flag=1;
       end

    end
  end
end %end if Nref(z)>1 & unwrap_istep<5

if exist('Rinv.mat','file')
  delete Rinv.mat
end

fclose('all');
close all;
```